Программирование на Python для Геоскан Пионер

Учебная глава и практикум

Курс «Программные средства БПЛА»

Корягин Сергей Андреевич
асп. СПбГЭТУ «ЛЭТИ», педагог ЦО «Кудрово»

г. Кудрово, 2026

Contents

1 Глава «Программирование на Python»: цели и маршрут 3
- 1.1 Карта исходных скриптов . . . 3
- 1.2 Дидактическая лестница . . . 3
- 1.3 Минимальные требования к входу в раздел . . . 4

2 Уроки 1–2: первое подключение, арминг, взлёт и посадка 5
- 2.1 Цели уроков . . . 5
- 2.2 Ключевые команды . . . 5
- 2.3 Опорные листинги . . . 5
- 2.4 Критерии успешности . . . 7
- 2.5 Контрольные вопросы . . . 7

3 Урок 3: телеметрия и диагностическая индикация 8
- 3.1 Цели урока . . . 8
- 3.2 Необходимые знания . . . 8
- 3.3 Ключевые команды . . . 8
- 3.4 Опорные листинги . . . 8
- 3.5 Критерии успешности . . . 12

4 Уроки 4–5: полёт по прямой и движение по локальным точкам 13
- 4.1 Цели уроков . . . 13
- 4.2 Ключевые команды . . . 13
- 4.3 Опорные листинги . . . 13
- 4.4 Критерии успешности . . . 16

5 Урок 6: базовые фигуры и миссия по точкам 17
- 5.1 Цели урока . . . 17
- 5.2 Опорные листинги . . . 17
- 5.3 Формула окружности . . . 20
- 5.4 Критерии успешности . . . 21

6 Урок 7: интерактивное управление и запуск Lua-сценариев 22
- 6.1 Цели урока . . . 22
- 6.2 Опорные листинги . . . 22

6.3 Ключевые команды 25
6.4 Критерии успешности 25

7 Уроки 8–9: камера, кадры и калибровка 26
7.1 Цели уроков 26
7.2 Опорные листинги 26
7.3 Методика калибровки 32
7.4 Критерии успешности 32

8 Уроки 10–11: ArUco-метки и визуальная обратная связь 33
8.1 Цели уроков 33
8.2 Опорные листинги 33
8.3 Логика регулирования 40
8.4 Критерии успешности 40

9 Урок 12: трекинг человека и жестовое управление 41
9.1 Цели урока 41
9.2 Опорные листинги 41
9.3 Педагогический разбор 56
9.4 Критерии успешности 57

10 Уроки 13–14: многодронные сценарии и итоговый проект 58
10.1 Цели уроков 58
10.2 Опорные листинги 58
10.3 Итоговый проект 70
10.4 Контрольные вопросы 70

11 План-проспект главы по программированию на Python 71

12 Приложение 73
12.1 Алфавитный индекс ключевых команд 73
12.2 Карта скриптов для преподавателя 74

1 Глава «Программирование на Python»: цели и маршрут

Цели раздела

- Перейти от базовых команд arm/takeoff/land к автономным миссиям.
- Освоить два подхода к движению: по точкам и по скоростям.
- Научиться использовать телеметрию, камеру и базовые алгоритмы компьютерного зрения.
- Подготовить учащихся к проектам ArUco-навигации и трекинга человека.

1.1 Карта исходных скриптов

Группа скриптов	Учебная роль
mission_examples/*.py	Базовое пилотирование, точки, прямолинейный полёт, окружность
get_telemetry/*.py	Диагностика батареи, дальномера и локальной позиции
camera_examples/*.py	Поток, кадры, калибровка камеры
aruco_examples/*.py	Детекция маркеров, оценка позы, полёт по визуальному ориентиру
PioneerHumanTracking/*.py	Продвинутый CV: скелетная модель, жесты, контур управления
group_flight_examples/*.py	Многодронные миссии и синхронизация потоков

Table 1: Покрытие учебных тем по структуре репозитория

1.2 Дидактическая лестница

1. Безопасный старт: соединение, арминг, взлёт, посадка.
2. Контроль движения: скорость и локальные точки.
3. Фигуры полёта: прямоугольник, треугольник, окружность.

4. Сенсоры и обратная связь.
5. Компьютерное зрение.
6. Интеграционный проект.

Методический акцент

Последовательность заданий построена по принципу «одна новая идея за урок»: сначала только одна новая команда API, затем комбинация команд в мини-миссию.

1.3 Минимальные требования к входу в раздел

- Базовый синтаксис Python: переменные, циклы, функции, try/except.
- Навыки запуска скрипта и чтения консольного вывода.
- Понимание системы координат x,y,z,yaw.

Задание 1. Стартовая диагностика

Составьте таблицу из трёх команд API Pioneer, которые отвечают за запуск миссии, движение и безопасное завершение. Для каждой команды укажите один риск ошибочного применения.

2 Уроки 1–2: первое подключение, арминг, взлёт и посадка

2.1 Цели уроков

- Запускать базовый цикл управления без аварийного завершения.
- Понимать разницу между disarm() и land().
- Формировать безопасный блок try/finally.

2.2 Ключевые команды

- Pioneer()
- arm(), takeoff(), land(), disarm()
- close_connection()

2.3 Опорные листинги

Listing 1: Минимальное управление моторами и просмотром видеопотока

```
from pioneer_sdk import Pioneer, Camera
import cv2
import time

if __name__ == "__main__":
    print(
        """
    1 -- arm (завести моторы)
    2 -- disarm (остановить моторы)
    ESC -- выход
        """
    )

    pioneer_mini = Pioneer()
    camera = Camera()

    try:
        while True:
```

```
            frame = camera.get_cv_frame()
            if frame is not None:
                cv2.imshow("pioneer_camera_stream", frame)

            key = cv2.waitKey(1)
            if key == 27:  # ESC
                print("esc pressed")
                cv2.destroyAllWindows()
                pioneer_mini.disarm()
                break
            elif key == ord("1"):
                pioneer_mini.arm()
            elif key == ord("2"):
                pioneer_mini.disarm()

            time.sleep(0.02)

    finally:
        time.sleep(1)
        pioneer_mini.disarm()
        pioneer_mini.close_connection()
        del pioneer_mini
```

Listing 2: Взлёт, выход на точку и посадка

```
from pioneer_sdk import Pioneer
import time

# Инициализация дрона
pioneer = Pioneer(ip='192.168.137.4', mavlink_port=5656)

try:
    # Взлет на 1 метр
    pioneer.arm()
    pioneer.takeoff()
    pioneer.go_to_local_point(x=0, y=0, z=1, yaw=0)
    while not pioneer.point_reached():
        time.sleep(0.1)

    # Посадка
    pioneer.land()
    time.sleep(3)
```

```
    pioneer.disarm()

except KeyboardInterrupt:
    print("Остановка программы, производится посадка")
    pioneer.land()
    time.sleep(3)
    pioneer.disarm()
```

2.4 Критерии успешности

1. Дрон выполняет арминг и взлёт без ручного вмешательства.
2. После команды посадки двигатели корректно останавливаются.
3. При KeyboardInterrupt аварийный сценарий не оставляет дрон в воздухе.

Задание 2. Минимальный набор упражнений

1. Запустите test.py, проверьте команды 1, 2, ESC.
2. Запустите PiZero_test.py и зафиксируйте в журнале время от takeoff() до land().
3. Добавьте паузу перед disarm() и объясните, зачем она нужна.

2.5 Контрольные вопросы

1. Почему land() предпочтительнее disarm() при штатном завершении?
2. Что должно находиться в блоке finally?
3. Какие признаки устойчивого соединения вы видите в ходе запуска?

3 Урок 3: телеметрия и диагностическая индикация

3.1 Цели урока

- Считать базовую телеметрию в цикле и интерпретировать значения.
- Настроить «светофор» батареи и простую реакцию на высоту по дальномеру.

3.2 Необходимые знания

- Цикл while True, задержка time.sleep.
- Проверка None и пустых значений.

3.3 Ключевые команды

- get_battery_status(), get_dist_sensor_data(), get_local_position_lps()
- led_control(led_id=255, r, g, b)

3.4 Опорные листинги

Listing 3: Диагностика батареи и индикация

```
from pioneer_sdk import Pioneer
import time

# Создаем объект mini класса Pioneer
mini = Pioneer()

# Бесконечный цикл для постоянного опроса состояния батареи
while True:
    # Получаем текущий вольтаж батареи
    battery_voltage = mini.get_battery_status()

    # Проверяем, получены ли данные (не None)
    if battery_voltage is not None:
        # Выводим вольтаж батареи в консоль без округления
        print("Напряжение батареи:", battery_voltage, "В")

        # Менее 3.6 В - красный (низкий заряд)
```

```
        if battery_voltage < 3.6:
            mini.led_control(led_id=255, r=0.1, g=0, b=0)  # Красн
               ый цвет
        # От 3.6 В до 4.0 В - желтый (средний заряд)
        elif battery_voltage < 4.0:
            mini.led_control(led_id=255, r=0.1, g=0.1, b=0)  # Жел
               тый цвет (красный + зеленый)
        # От 4.0 В до 4.2 В - зеленый (хороший заряд)
        elif battery_voltage <= 4.2:
            mini.led_control(led_id=255, r=0, g=0.1, b=0)  # Зелен
               ый цвет

    # Задержка для уменьшения частоты запросов
    time.sleep(2)
```

Listing 4: Дальномер: высота/расстояние

```
from pioneer_sdk import Pioneer
import time

# Создаем объект mini класса Pioneer
mini = Pioneer()

# Бесконечный цикл для постоянного опроса дальномера
while True:
    # Получаем данные с дальномера (в метрах)
    distance = mini.get_dist_sensor_data()

    # Проверяем, получены ли данные (не None)
    if distance is not None:
        # Выводим расстояние в консоль
        print(f'Расстояние: {distance:.2f} м')

    # Задержка вывода сообщений в консоль
    time.sleep(2)
```

Listing 5: Локальная позиция в LPS

```
from pioneer_sdk import Pioneer
import time

# Создаем объект mini кдасса Pioneer
mini = Pioneer()
```

```
# Бесконечный цикл для постоянного опроса позиции дрона
while True:
    # Получаем текущие координаты дрона (метод возвращает координа
        ты в виде списка)
    array_of_coordinates = mini.get_local_position_lps()

    # Проверяем, что координаты были получены (не пустой список ил
        и None)
    if array_of_coordinates:
        # Выводим координаты x, y, z в консоль
        print(f'x={array_of_coordinates[0]} , y={
            array_of_coordinates[1]}, z={array_of_coordinates[2]}')

        # Пример вывода координаты x отдельно
        # print(f'x={array_of_coordinates[0]}')

    # Задержка между итерациями
    time.sleep(2)
```

Listing 6: Изменение цвета LED по высоте

```
from pioneer_sdk import Pioneer
import time

# Определяем временной интервал обновления данных
delta_time = 0.1

# Инициализируем переменные для цвета светодиодов (RGB)
r = 0
g = 0
b = 0

# Определяем пороговые значения высоты для смены цвета
height_1 = 0.25
height_2 = 0.5
height_3 = 0.75

#   <------- height_1 ------- height_2 ------- height_3 ------->
#    красный            зеленый             синий             белый

if __name__ == "__main__":                          # Проверяем, что ск
```

```
рипт запущен как основной файл
 pioneer_mini = Pioneer(logger=False)        # Создаем объект уп
    равления дроном, отключая логирование
 curr_time = time.time()                     # Фиксируем текущее
     время

 try:
     while True:                                                 #
        Запускаем бесконечный цикл
         if time.time() - curr_time > delta_time:                #
            Проверяем, прошло ли заданное время (delta_time)
             tof_data = pioneer_mini.get_dist_sensor_data()  #
                Получаем данные с датчика расстояния
             print(tof_data)                                     #
                Выводим данные в консоль для мониторинга

             if tof_data is not None:                            #
                Проверяем, что данные успешно получены с датчик
                а

                 # Определяем цвет светодиодов в зависимости от
                     высоты
                 if tof_data <= height_1:
                     r = 0.1  # Красный
                     g = 0
                     b = 0
                 elif height_1 < tof_data <= height_2:
                     r = 0
                     g = 0.1  # Зеленый
                     b = 0
                 elif height_2 < tof_data <= height_3:
                     r = 0
                     g = 0
                     b = 0.1  # Синий
                 elif tof_data >= height_3:
                     r = 0.1
                     g = 0.1
                     b = 0.1  # Белый

                 # Управляем светодиодами дрона
                 pioneer_mini.led_control(led_id=255, r=r, g=g,
```

```
                        b=b)

                    # Обновляем время последнего обновления
                    curr_time = time.time()

    except KeyboardInterrupt:
        print("Остановка программы. Выключаем светодиоды...")
        pioneer_mini.led_control(led_id=255, r=0, g=0, b=0)  # Вык
           лючаем светодиоды

    finally:
        pioneer_mini.close_connection()                          # Зак
           рываем соединение с дроном
```

Инженерная интерпретация

- Телеметрия в учебных задачах нужна не для «красивого вывода», а для принятия решений.
- LED-индикация — простой способ дебага без подключения терминала.

Задание 3. Минимальный набор упражнений

1. Измените пороги напряжения в GET_battery_status.py и опишите ожидаемое поведение.
2. Добавьте в GET_dist_sensor_data.py вывод предупреждения при расстоянии меньше 0.3 м.
3. В GET_local_position.py добавьте вычисление модуля вектора позиции.

3.5 Критерии успешности

1. Скрипт работает не менее 1 минуты без зависаний.
2. При потере значения датчика в консоль не выводятся ошибки трассировки.
3. Индикация LED соответствует выбранным порогам.

4 Уроки 4–5: полёт по прямой и движение по локальным точкам

4.1 Цели уроков

- Освоить управление через скорости set_manual_speed.
- Освоить навигацию по точкам go_to_local_point.
- Научиться ждать завершение этапа через point_reached().

4.2 Ключевые команды

- set_manual_speed(vx, vy, vz, yaw_rate)
- go_to_local_point(x, y, z, yaw)
- point_reached()

4.3 Опорные листинги

Listing 7: Прямолинейное движение по скорости

```
from pioneer_sdk import Pioneer
import time

if __name__ == "__main__":
    print("start")                          # Выводим сообщение о начале
        выполнения программы
    pioneer_mini = Pioneer()                # Создаем объект для управле
       ния дроном

    try:
        pioneer_mini.arm()                  # Включаем моторы дрона
        pioneer_mini.takeoff()              # Взлет
        time.sleep(1)                       # Ждем 1 секунду после взлет
            а

        # Дрон летит вперед (по оси Y) в течение 2 секунд со скоро
            стью 1 м/с
        t = time.time()
```

```
        while True:
                                                                                #
            Отправка команды set_manual_speed всегда осуществляется
             в бесконечном цикле
             pioneer_mini.set_manual_speed(vx=0, vy=1, vz=0,
                yaw_rate=0)  # Устанавливаем скорость движения по о
                си y
             time.sleep(0.05)
                                                                        # Небол
                ышая задержка
             if time.time() - t > 2:
                                                                # Проверяем, п
                рошло ли 2 секунды
                 break

                    # Если прошло - выход из цикла

        time.sleep(2)
                                                                        # Жд
            ем 2 секунды перед следующим движением

        # Дрон летит вправо (по оси X) в течение 2 секунд со скоро
            стью 1 м/с
        t = time.time()
                                                                        # Запо
            минаем текущее время
        while True:
             pioneer_mini.set_manual_speed(vx=1, vy=0, vz=0,
                yaw_rate=0)  # Устанавливаем скорость движения по о
                си x
             time.sleep(0.05)
             if time.time() - t > 2:
                 break

        time.sleep(2)
                                                                        # Жд
            ем 2 секунды перед посадкой

        pioneer_mini.land()
                                                                # Осуществ
            ляем посадку
```

```
    except KeyboardInterrupt:
        print("Остановка программы, производится посадка")
        pioneer_mini.land()
                                                        # При прер
           ывании программы дрон автоматически садится

    finally:
        pioneer_mini.close_connection()
                                                # Закрываем соединение
            с дроном
        del pioneer_mini
                                                        # Удаля
           ем объект дрона
```

Listing 8: Переходы по локальным точкам

```
from pioneer_sdk import Pioneer
import time

# Инициализация дрона
pioneer = Pioneer()

try:
    # Взлет на 1 метр
    pioneer.arm()
    pioneer.takeoff()
    pioneer.go_to_local_point(x=0, y=0, z=1, yaw=0)
    while not pioneer.point_reached():
        time.sleep(0.1)

    # Полет в первую точку
    pioneer.go_to_local_point(x=0, y=1, z=1, yaw=0)
    while not pioneer.point_reached():
        time.sleep(0.1)

    # Полет во вторую точку
    pioneer.go_to_local_point(x=1, y=1, z=1, yaw=0)
    while not pioneer.point_reached():
        time.sleep(0.1)

    # Посадка
```

```
    pioneer.land()

except KeyboardInterrupt:
    print("Остановка программы, производится посадка")
    pioneer.land()
```

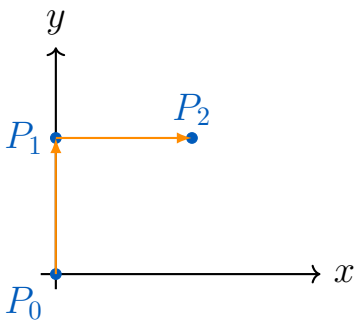

Figure 1: Схема маршрута из go_to_local_point.py

Задание 4. Минимальный набор упражнений

1. Измените длительность первого участка в set_manual_speed.py с 2 до 1 секунды.
2. Добавьте третью точку в go_to_local_point.py.
3. Сделайте разворот на 90° в последней точке.

4.4 Критерии успешности

1. Прямолинейный участок выполняется без заметного дрейфа.
2. Каждая точка подтверждается point_reached().
3. Посадка выполняется в пределах безопасной зоны.

5 Урок 6: базовые фигуры и миссия по точкам

5.1 Цели урока

- Сформировать миссию из массива точек.
- Понять параметризацию окружности через угол и радиус.
- Совместить движение и видеонаблюдение.

5.2 Опорные листинги

Listing 9: Маршрут по массиву точек с кадрами камеры

```
from pioneer_sdk import Pioneer,Camera
import cv2
import math

# Инициализация дрона
mini = Pioneer()
camera = Camera()

# Функция перевода радианов в градусы
def degrees_to_radians(deg):
    return math.radians(deg)

# Функция вывода видео с камеры дрона
def show_camera():
        frame = camera.get_cv_frame()
        if frame is not None:
            cv2.imshow("pioneer_camera", frame)
            cv2.waitKey(1)

# Функция взлета на высоту 1 метр
def go_to_start_point():
    mini.arm()
    mini.takeoff()
    mini.go_to_local_point(x=0, y=0, z=1, yaw=0)  # Подъем на 1 ме
        тр
    while not mini.point_reached():
        show_camera()
```

```
# Функция полета по точкам
def fly_through_points(points):
    for point in points:
        mini.go_to_local_point(x=point['x'], y=point['y'], z=point
            ['z'], yaw=degrees_to_radians(point['yaw']))
        while not mini.point_reached():
            show_camera()

# Массив точек
waypoints = [
    {'x': 1, 'y': 0, 'z': 0.7, 'yaw': 0},    # Точка 1
    {'x': 1, 'y': 1, 'z': 0.7, 'yaw': 0},    # Точка 2
    {'x': 0, 'y': 1, 'z': 0.7, 'yaw': 0},    # Точка 3
    {'x': 0, 'y': 0, 'z': 0.7, 'yaw': 0},    # Возврат к начальной
        точке
]

try:
    go_to_start_point()              # Взлет на 1 м
    fly_through_points(waypoints)    # Полет по точкам
    mini.land()                      # Посадка
except KeyboardInterrupt:
    print('Остановка программы, производится посадка')
    mini.land()
finally:
    mini.land()
```

Listing 10: Окружность по дискретным точкам

```
from pioneer_sdk import Pioneer, Camera
import cv2
import math
import numpy as np

# Настройки полёта
flight_height = 1.0            # Высота в метрах
circle_radius = 0.6            # Радиус окружности
number_of_points = 10          # Кол-во точек на окружности
angle_step = 360 / number_of_points

# Начальные параметры
```

```
current_angle = 0
first_point = True
mission_complete = False

# Функция расчёта координат на окружности
def get_point_on_circle(angle_degrees, radius):
    angle_radians = math.radians(angle_degrees)
    x = radius * math.cos(angle_radians)
    y = radius * math.sin(angle_radians)
    return x, y

# Функция вывода видео с камеры дрона
def show_camera_frame():
        frame = camera.get_cv_frame()
        if frame is not None:
            cv2.imshow("pioneer_camera", frame)
            cv2.waitKey(1)

if __name__ == "__main__":
    print("Старт полёта по окружности")

    # Инициализация дрона и камеры
    drone = Pioneer()
    camera = Camera()

    try:
        drone.arm()
        drone.takeoff()

        while True:
            show_camera_frame()

            # Полет к следующей точке
            if drone.point_reached() or first_point:
                first_point = False

                if current_angle >= 360:
                    mission_complete = True
                else:
                    x, y = get_point_on_circle(current_angle,
                        circle_radius)
```

```
                    yaw = math.radians(current_angle)

                    drone.go_to_local_point(x=x, y=y, z=
                        flight_height, yaw=yaw)
                    current_angle += angle_step

            # Завершение миссии при нажатии ESC (27)
            if cv2.waitKey(1) == 27 or mission_complete:
                print("Миссия завершена или нажата клавиша ESC")
                drone.land()
                break

    except KeyboardInterrupt:
        print("Принудительное завершение программы")
        drone.land()

    finally:
        # Всегда выполняется при завершении
        print("Посадка дрона и завершение программы")
        cv2.destroyAllWindows()
        drone.land()
        drone.close_connection()
```

5.3 Формула окружности

$$x = r\cos\varphi, \quad y = r\sin\varphi$$

где r — радиус, φ — текущий угол в радианах.

φ	x/r	y/r	Интерпретация
0	1.00	0.00	старт на оси x
$\pi/2$	0.00	1.00	верхняя точка
π	-1.00	0.00	противоположная точка
$3\pi/2$	0.00	-1.00	нижняя точка

Table 2: Опорные значения для генерации фигуры

Задание 5. Минимальный набор упражнений

1. Увеличьте number_of_points до 16 и сравните плавность траектории.
2. Постройте квадратный маршрут из массива waypoints.
3. Добавьте возврат в стартовую точку на высоте 1 м перед посадкой.

5.4 Критерии успешности

1. Фигура замыкается без резкого рывка на последнем участке.
2. Камера непрерывно показывает кадры в ходе миссии.
3. Все этапы завершены с безопасной посадкой.

6 Урок 7: интерактивное управление и запуск Lua-сценариев

6.1 Цели урока

- Понять, как устроен ручной канал управления через клавиатуру.
- Освоить отправку RC-каналов и логику привязки клавиш.
- Научиться запускать и останавливать Lua-сценарий с Python.

6.2 Опорные листинги

Listing 11: Ручное управление через WASD и RC-каналы

```
from pioneer_sdk import Pioneer, Camera
import cv2
import numpy as np
import time

if __name__ == "__main__":
    print(
        """
    1 -- arm
    2 -- disarm
    3 -- takeoff
    4 -- land

    ↺q  w↑  e↻     i-↑
    ←a      d→     k-↓
        s↓"""
    )
    pioneer_mini = Pioneer()
    camera = Camera()
    min_v = 1300
    max_v = 1700
    try:
        while True:
            ch_1 = 1500
            ch_2 = 1500
            ch_3 = 1500
```

```
            ch_4 = 1500
            ch_5 = 2000
            frame = camera.get_frame()
            if frame is not None:
                camera_frame = cv2.imdecode(
                    np.frombuffer(frame, dtype=np.uint8), cv2.
                        IMREAD_COLOR
                )
                cv2.imshow("pioneer_camera_stream", camera_frame)
            key = cv2.waitKey(1)
            if key == 27:  # esc
                print("esc pressed")
                cv2.destroyAllWindows()
                pioneer_mini.land()
                break
            elif key == ord("1"):
                pioneer_mini.arm()
            elif key == ord("2"):
                pioneer_mini.disarm()
            elif key == ord("3"):
                time.sleep(2)
                pioneer_mini.arm()
                time.sleep(1)
                pioneer_mini.takeoff()
                time.sleep(2)
            elif key == ord("4"):
                time.sleep(2)
                pioneer_mini.land()
                time.sleep(2)
            elif key == ord("w"):
                ch_3 = min_v
            elif key == ord("s"):
                ch_3 = max_v
            elif key == ord("a"):
                ch_4 = min_v
            elif key == ord("d"):
                ch_4 = max_v
            elif key == ord("q"):
                ch_2 = 2000
            elif key == ord("e"):
                ch_2 = 1000
```

```
            elif key == ord("i"):
                ch_1 = 2000
            elif key == ord("k"):
                ch_1 = 1000

            pioneer_mini.send_rc_channels(
                channel_1=ch_1,
                channel_2=ch_2,
                channel_3=ch_3,
                channel_4=ch_4,
                channel_5=ch_5,
            )
            time.sleep(0.02)
    finally:
        time.sleep(1)
        pioneer_mini.land()

        pioneer_mini.close_connection()
        del pioneer_mini
```

Listing 12: Управление Lua-скриптом на борту

```
from pioneer_sdk import Pioneer

if __name__ == "__main__":
    pioneer_mini = Pioneer()  # Создаем объект управления дроном

    while True:            # Запускаем бесконечный цикл для обработ
       ки команд пользователя
        cmd = input()      # Считываем ввод пользователя

        if cmd == "start":
            pioneer_mini.lua_script_control("Start")  # Отправляем
                команду на запуск Lua-скрипта на дроне

        elif cmd == "stop":
            pioneer_mini.lua_script_control("Stop")   # Отправляем
                команду на остановку Lua-скрипта
```

6.3 Ключевые команды

- send_rc_channels(channel_1,...,channel_5)
- lua_script_control("Start") и lua_script_control("Stop")
- cv2.waitKey(1)

Внимание

Интерактивные скрипты подходят для тренировки пилотирования и диагностики, но не заменяют автономные безопасные миссии.

Задание 6. Минимальный набор упражнений

1. Измените шаг каналов min_v/max_v и проверьте чувствительность.
2. Добавьте клавишу экстренной остановки с land().
3. Реализуйте текстовую подсказку активной команды в консоли.

6.4 Критерии успешности

1. Все горячие клавиши вызывают ожидаемую реакцию.
2. При выходе через ESC соединение закрывается корректно.
3. Lua-скрипт запускается и останавливается по командам без перезапуска Python.

7 Уроки 8–9: камера, кадры и калибровка

7.1 Цели уроков

- Получать видеопоток с дрона в формате OpenCV.
- Сохранять кадры и подготавливать данные для CV-задач.
- Выполнять калибровку камеры и сохранять коэффициенты в data.yml.

7.2 Опорные листинги

Listing 13: Запуск и остановка видеопотока

```
from pioneer_sdk import VideoStream

if __name__ == "__main__":
    stream = VideoStream()        # Создаем объект потока видео

    while True:                   # Бесконечный цикл, ожидающий кома
        нд
        cmd = input()             # Ожидаем ввода команды от пользов
            ателя в консоль

        if cmd == "start":        # Если введена команда 'start'
            stream.start()        # Запускаем видеопоток

        elif cmd == "stop":       # Если введена команда 'stop'
            stream.stop()         # Останавливаем видеопоток
```

Listing 14: Получение кадров с камеры

```
import cv2
import numpy as np
from pioneer_sdk import Camera

# Создаем объект управления камерой
camera = Camera()

if __name__ == "__main__":
    while True:
        frame = camera.get_cv_frame()    # Получаем изображение с к
            амеры
```

```
        if frame is not None:            # Проверяем, что изображен
            ие получено
            cv2.imshow("video", frame)  # Отображаем кадр в окне с
                названием "video"

        if cv2.waitKey(1) == 27:  # Выход из цикла при нажатии кла
            виши ESC (код 27)
            break

    cv2.destroyAllWindows()        # Закрываем все открытые окна
        OpenCV
```

Listing 15: Калибровка камеры по шахматной доске

```
#!/usr/bin/python3

"""
Performs camera calibration using checkerboard pattern. Saves
    calibration
values into a file.
More: https://docs.opencv.org/4.x/dc/dbb/tutorial_py_calibration.
    html
"""

import cv2
import numpy as np
import glob
import time
from pioneer_sdk import Camera


def get_images_from_folder(folder_name, file_type="*.jpg"):
    """
    Retreive images from a directory using a provided path (`
        folder_name`)
    """
    images_list = glob.glob(folder_name + "/" + file_type)
    images = []
    for fname in images_list:
        img = cv2.imread(fname)
        images.append(img)
    return images
```

```python
def get_images_from_drone_camera(camera, save=False):
    """
    Retreive images from a drone's camera
    """
    images = []
    print("Press '1' to take frame\nPress ESC or 'q' to finish")
    while True:
        frame = camera.get_cv_frame()
        cv2.imshow("frames", frame)

        key = cv2.waitKey(1)
        if key == 27 or key == ord("q"):
            cv2.destroyAllWindows()
            return images
        if key == ord("1"):
            images.append(frame)
            print("Frame #" + str(len(images)))
            if save:
                cv2.imwrite("frame_" + str(len(images)) + ".jpg",
                    frame)
            time.sleep(1)


def get_images_from_computer_camera(capture, save=False):
    """
    Retreive images using the PC's camera device
    """
    images = []
    print("Press '1' to take frame\nPress ESC or 'q' to finish")
    while True:
        ret, frame = capture.read()
        cv2.imshow("frames", frame)

        key = cv2.waitKey(1)
        if key == 27 or key == ord("q"):
            cv2.destroyAllWindows()
            return images
        if key == ord("1"):
            images.append(frame)
```

```
            print("Frame #" + str(len(images)))
            if save:
                cv2.imwrite("frame_" + str(len(images)) + ".jpg",
                    frame)
            time.sleep(1)


def calibrate(images):
    # Defining the dimensions of checkerboard
    CHECKERBOARD = (6, 9)
    criteria = (cv2.TERM_CRITERIA_EPS + cv2.TERM_CRITERIA_MAX_ITER
        , 30, 0.001)

    # Creating vector to store vectors of 3D points for each
        checkerboard image
    objpoints = []
    # Creating vector to store vectors of 2D points for each
        checkerboard image
    imgpoints = []

    # Defining the world coordinates for 3D points
    objp = np.zeros((1, CHECKERBOARD[0] * CHECKERBOARD[1], 3), np.
        float32)
    objp[0, :, :2] = np.mgrid[0 : CHECKERBOARD[0], 0 :
        CHECKERBOARD[1]].T.reshape(-1, 2)

    # Extracting path of individual image stored in a given
        directory
    for img in images:
        gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
        # Find the chess board corners
        # If desired number of corners are found in the image then
            ret = true
        ret, corners = cv2.findChessboardCorners(
            gray,
            CHECKERBOARD,
            cv2.CALIB_CB_ADAPTIVE_THRESH
            + cv2.CALIB_CB_FAST_CHECK
            + cv2.CALIB_CB_NORMALIZE_IMAGE,
        )
```

```
        if ret == True:
            objpoints.append(objp)
            # refining pixel coordinates for given 2d points.
            corners2 = cv2.cornerSubPix(gray, corners, (11, 11),
                (-1, -1), criteria)

            imgpoints.append(corners2)

            # Draw and display the corners
            img = cv2.drawChessboardCorners(img, CHECKERBOARD,
                corners2, ret)

        cv2.imshow("img", img)
        cv2.waitKey(0)

    cv2.destroyAllWindows()

    ret, mtx, dist, rvecs, tvecs = cv2.calibrateCamera(
        objpoints, imgpoints, gray.shape[::-1], None, None
    )

    return mtx, dist


def save_coefficients(mtx, dist, path):
    """Save the camera matrix and the distortion coefficients to
       given path/file."""
    cv_file = cv2.FileStorage(path, cv2.FILE_STORAGE_WRITE)
    cv_file.write("mtx", mtx)
    cv_file.write("dist", dist)
    # note you *release* you don't close() a FileStorage object
    cv_file.release()


def load_coefficients(path):
    """Loads camera matrix and distortion coefficients."""
    # FILE_STORAGE_READ
    cv_file = cv2.FileStorage(path, cv2.FILE_STORAGE_READ)

    # note we also have to specify the type to retrieve other wise
        we only get a
```

```
    # FileNode object back instead of a matrix
    camera_matrix = cv_file.getNode("mtx").mat()
    dist_coeffs = cv_file.getNode("dist").mat()

    cv_file.release()
    return camera_matrix, dist_coeffs


def main():
    """
    Acquires images from the provided image source, calibrates the
        camera.

    This particular example uses the drone's camera as the image
       source.
    However, you can change the source by replacing
    `get_images_from_computer_camera` w/ another function.

    Examples:
    ```
    # Load images from a directory
    images = get_images_from_folder('images')
    # ...
    ```

    ```
    # Capture the images from a camera device
    capture = cv2.VideoCapture(0)
    images = get_images_from_computer_camera(capture)
    capture.release()
    # ...
    ```
    """

    # Acquire images from the drone's camera
    camera = Camera()
    images = get_images_from_drone_camera(camera)

    mtx, dist = calibrate(images)

    save_coefficients(mtx, dist, "data.yml")
```

```
if __name__ == "__main__":
    main()
```

7.3 Методика калибровки

1. Снять 10–15 кадров шахматной доски под разными углами.
2. Убедиться, что углы доски распознаются на большинстве кадров.
3. Сохранить матрицу камеры и коэффициенты дисторсии.
4. Повторно использовать data.yml в задачах ArUco.

Задание 7. Минимальный набор упражнений

1. Подготовьте набор из 12 кадров для калибровки.
2. Сохраните коэффициенты и проверьте, что файл data.yml создан.
3. Объясните, почему без калибровки оценка расстояния до маркера нестабильна.

7.4 Критерии успешности

1. Скрипт калибровки завершается без ошибок OpenCV.
2. В data.yml записаны mtx и dist.
3. При повторном запуске CV-скриптов файл корректно загружается.

8 Уроки 10–11: ArUco-метки и визуальная обратная связь

8.1 Цели уроков

- Детектировать ArUco-метки в видеопотоке.
- Оценивать положение маркера через solvePnP.
- Управлять дроном на основе визуальных признаков.

8.2 Опорные листинги

Listing 16: Базовая детекция ArUco

```
import cv2
from pioneer_sdk import Camera

# Dictionary of aruco-markers
aruco_dict = cv2.aruco.getPredefinedDictionary(cv2.aruco.
    DICT_6X6_50)
# Parameters for marker detection (in this case, default
    parameters)
aruco_params = cv2.aruco.DetectorParameters()
# Create instance of ArucoDetector.
# Required starting from version opencv 4.7.0
aruco_detector = cv2.aruco.ArucoDetector(aruco_dict, aruco_params)

# Connect to the drone camera
camera = Camera()

if __name__ == "__main__":
    while True:
        frame = camera.get_cv_frame()  # Get frame (in this case
            already decoded)
        # Detect markers
        corners, ids, rejected = aruco_detector.detectMarkers(
            frame)
        # Highlight the decoded markers on the image
        cv2.aruco.drawDetectedMarkers(frame, corners, ids)
        cv2.imshow("video", frame)  # Show an image on the screen
```

```
        if cv2.waitKey(1) == 27:  # Exit if the ESC key is pressed
            break

    cv2.destroyAllWindows()  # Close all opened openCV windows
```

Listing 17: Координаты и оси маркера

```
import cv2
import numpy as np
from pioneer_sdk import Camera


def load_coefficients(path):
    cv_file = cv2.FileStorage(path, cv2.FILE_STORAGE_READ)

    camera_matrix = cv_file.getNode("mtx").mat()
    dist_coeffs = cv_file.getNode("dist").mat()

    cv_file.release()
    return camera_matrix, dist_coeffs


# Dictionary of aruco-markers
aruco_dict = cv2.aruco.getPredefinedDictionary(cv2.aruco.
   DICT_6X6_50)
# Parameters for marker detection (in this case, default
   parameters)
aruco_params = cv2.aruco.DetectorParameters()
# Create instance of ArucoDetector.
# Required starting from version opencv 4.7.0
aruco_detector = cv2.aruco.ArucoDetector(aruco_dict, aruco_params)

# Load camera matrix and distortion coefficients from file
camera_matrix, dist_coeffs = load_coefficients("data.yml")

size_of_marker = 0.05  # side length in meters

# Coordinates of marker corners
points_of_marker = np.array(
    [
        (size_of_marker / 2, -size_of_marker / 2, 0),
```

```
        (-size_of_marker / 2, -size_of_marker / 2, 0),
        (-size_of_marker / 2, size_of_marker / 2, 0),
        (size_of_marker / 2, size_of_marker / 2, 0),
    ]
)

# Connect to the drone camera

if __name__ == "__main__":
    camera = Camera()
    while True:
        frame = camera.get_cv_frame()  # Get frame (in this case
            already decoded)
        if frame is not None:
            # Detect markers
            corners, ids, rejected = aruco_detector.detectMarkers(
                frame)
            if corners:
                # Calculate pose for first detected marker
                success, rvecs, tvecs = cv2.solvePnP(
                    points_of_marker, corners[0], camera_matrix,
                        dist_coeffs
                )
                print(success, tvecs)
                # Draw axes
                cv2.drawFrameAxes(frame, camera_matrix,
                    dist_coeffs, rvecs, tvecs, 0.1)
            cv2.imshow("video", frame)  # Show an image on the
                screen

        if cv2.waitKey(1) == 27:  # Exit if the ESC key is pressed
            break

    cv2.destroyAllWindows()  # Close all opened openCV windows
```

Listing 18: Полёт с визуальной стабилизацией по маркеру

```
import cv2
from pioneer_sdk import Pioneer, Camera
import numpy as np
import threading
import time
```

```python
from collections import deque


def load_coefficients(path):
    cv_file = cv2.FileStorage(path, cv2.FILE_STORAGE_READ)
    camera_matrix = cv_file.getNode("mtx").mat()
    dist_coeffs = cv_file.getNode("dist").mat()
    cv_file.release()
    return camera_matrix, dist_coeffs


class VideoProcessingThread(threading.Thread):
    def __init__(self, camera_matrix, dist_coeffs):
        super().__init__()
        self.camera_matrix = camera_matrix
        self.dist_coeffs = dist_coeffs
        self.running = True
        self.latest_coordinates = None
        self.x_center = None
        self.frame = None

        self.frame_times = deque(maxlen=30)
        self.last_fps_log_time = time.time()

        self.aruco_dict = cv2.aruco.getPredefinedDictionary(cv2.
            aruco.DICT_4X4_50)
        self.aruco_params = cv2.aruco.DetectorParameters()
        self.aruco_detector = cv2.aruco.ArucoDetector(self.
            aruco_dict, self.aruco_params)

        self.size_of_marker = 0.1
        self.points_of_marker = np.array([
            (self.size_of_marker / 2, -self.size_of_marker / 2, 0)
                ,
            (-self.size_of_marker / 2, -self.size_of_marker / 2,
                0),
            (-self.size_of_marker / 2, self.size_of_marker / 2, 0)
                ,
            (self.size_of_marker / 2, self.size_of_marker / 2, 0),
        ])
```

```
        self.camera = Camera()

    def run(self):
        while self.running:
            try:
                frame = self.camera.get_cv_frame()
                corners, ids, _ = self.aruco_detector.
                    detectMarkers(frame)

                if ids is not None and len(corners) > 0:
                    x_center = int(sum([p[0] for p in corners
                        [0][0]]) / 4)
                    y_center = int(sum([p[1] for p in corners
                        [0][0]]) / 4)
                    dot_size = 5
                    frame[y_center - dot_size:y_center + dot_size,
                        x_center - dot_size:x_center + dot_size] =
                        [0, 0, 255]
                    cv2.aruco.drawDetectedMarkers(frame, corners)

                    success, rvecs, tvecs = cv2.solvePnP(
                        self.points_of_marker, corners[0], self.
                            camera_matrix, self.dist_coeffs
                    )
                    if success:
                        self.latest_coordinates = [tvecs.item(0),
                            tvecs.item(1), tvecs.item(2)]
                        self.x_center = x_center
                    else:
                        self.latest_coordinates = None
                        self.x_center = None
                else:
                    self.latest_coordinates = None
                    self.x_center = None

                self.frame = frame.copy()

                now = time.time()
                self.frame_times.append(now)
                if now - self.last_fps_log_time > 2.0 and len(self
                    .frame_times) > 1:
```

```
                    fps = len(self.frame_times) / (self.
                        frame_times[-1] - self.frame_times[0])
                    print(f"[Video Thread] FPS: {fps:.2f}")
                    self.last_fps_log_time = now

            except cv2.error as e:
                print(f"[Video Thread] OpenCV error: {e}")
                continue

    def stop(self):
        self.running = False


if __name__ == "__main__":
    camera_matrix, dist_coeffs = load_coefficients("data.yml")
    video_thread = VideoProcessingThread(camera_matrix,
        dist_coeffs)
    video_thread.start()

    mini = Pioneer()
    send_manual_speed = False
    airborne = False

    try:
        while True:
            coordinates = video_thread.latest_coordinates
            x_center = video_thread.x_center
            frame = video_thread.frame

            v_y = 0
            yaw_rate = 0

            if airborne and coordinates is not None and x_center
                is not None:
                distance = np.linalg.norm(coordinates)
                print(f"Distance: {distance:.2f} m")

                if distance > 1.5:
                    v_y = 0.4
                elif distance < 1:
                    v_y = -0.4
```

```
            if x_center < frame.shape[1] / 3:
                yaw_rate = -0.4
            elif x_center > frame.shape[1] * 2 / 3:
                yaw_rate = 0.4

        # Если маркер виден и есть команды движения
        if airborne and (v_y != 0 or yaw_rate != 0):
            mini.set_manual_speed_body_fixed(vx=0, vy=v_y, vz
               =0, yaw_rate=yaw_rate)
            send_manual_speed = True

        # Если маркер потерян, и до этого коптер двигался — ос
           тановка
        elif airborne and send_manual_speed:
            print("[INFO] Marker lost. Stopping drone.")
            mini.go_to_local_point_body_fixed(x=0, y=0, z=0,
               yaw=0)
            send_manual_speed = False

        if frame is not None:
            cv2.imshow("marker_detection", frame)

        key = cv2.waitKey(1)
        if key == 27:  # Esc
            break
        elif key == 32 and not airborne:  # Space
            print("[INFO] Takeoff initiated.")
            mini.arm()
            mini.takeoff()
            mini.go_to_local_point(x=0, y=0, z=1.5, yaw=0)
            while not mini.point_reached():
                time.sleep(0.1)
            airborne = True
            print("[INFO] Drone reached hover point.")

finally:
    print("Landing...")
    video_thread.stop()
    video_thread.join()
    cv2.destroyAllWindows()
```

```
        if airborne:
            mini.land()
        mini.close_connection()
        del mini
```

8.3 Логика регулирования

- По дальности до маркера корректируется скорость vy.
- По положению центра маркера в кадре корректируется yaw_rate.
- При потере маркера выполняется остановка команды движения.

Figure 2: Разделение кадра для управления рысканьем

Задание 8. Минимальный набор упражнений

1. Настройте пороги дальности 1.0–1.5 м под свою площадку.
2. Измените размер метки в size_of_marker и оцените влияние на координаты.
3. Добавьте вывод текущего режима: approach, hold, retreat.

8.4 Критерии успешности

1. Маркер обнаруживается устойчиво в 8 из 10 попыток.
2. Команды поворота соответствуют положению маркера в кадре.
3. При потере маркера дрон останавливается без дрейфа.

9 Урок 12: трекинг человека и жестовое управление

9.1 Цели урока

- Использовать MediaPipe Pose для выделения скелета.
- Преобразовывать относительные координаты в пиксели кадра.
- Управлять дроном по жестам и по ПД-регулятору.

9.2 Опорные листинги

Listing 19: Локальная камера и видеопоток для трекинга

```
import threading
import time

import cv2
import numpy as np
import socket


class Camera:

    def __init__(self, timeout=0.5, ip='192.168.4.1', port=8888,
        video_buffer_size=65000, log_connection=True):
        self.ip = ip
        self.port = port
        self.timeout = timeout
        self.VIDEO_BUFFER_SIZE = video_buffer_size
        self.tcp = None
        self.udp = None
        self._video_frame_buffer = None
        self.raw_video_frame = None
        self.connected = None
        self.log_connection = log_connection
        self._thread_stop = threading.Event()
        self._thread_stop.set()

        for i in range(15):
            if not self.connected:
                self.connected = self.reconnect()
```

```
                if self.log_connection:
                    print('Camera CONNECTED')
                break
            else:
                if self.log_connection:
                    print('Camera DISCONNECTED')
        self.connect()
        time.sleep(2)

    def new_tcp(self):
        """Returns new TCP socket"""
        sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
        sock.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
        sock.settimeout(self.timeout)
        return sock

    def new_udp(self):
        """Returns new UDP socket"""
        sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
        sock.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
        sock.settimeout(self.timeout)
        return sock

    def connect(self):
        self._getting_frames_tread = threading.Thread(target=self.
            _getting_frames, daemon=True)
        self._thread_stop.clear()
        self._getting_frames_tread.start()

    def disconnect(self):
        """Disconnect."""
        self._thread_stop.set()
        self.connected = False
        if self.tcp is not None:
            self.tcp.close()
            self.tcp = None
        if self.udp is not None:
            self.udp.close()
            self.udp = None
        if self.log_connection:
            print('Camera DISCONNECTED')
```

```
    def reconnect(self):
        """Connect to TCP and UDP sockets. Creates new ones if
           necessary."""
        if self.tcp is not None:
            self.tcp.close()
            self.tcp = None
        if self.udp is not None:
            self.udp.close()
            self.udp = None
        self.tcp = self.new_tcp()
        self.udp = self.new_udp()
        try:
            self.tcp.connect((self.ip, self.port))
            self.udp.bind(self.tcp.getsockname())
        except:
            return False
        return True

    def _getting_frames(self):
        while True:
            if self._thread_stop.is_set():
                break
            if not self.connected:
                while not self.reconnect():
                    pass
                self.connected = True
                if self.log_connection:
                    print('Camera CONNECTED')
            try:
                self._video_frame_buffer = self.udp.recv(self.
                   VIDEO_BUFFER_SIZE)
                end = self._video_frame_buffer.rfind(b'\xff\xd9')
                if end == -1:
                    continue
                self._video_frame_buffer = self.
                   _video_frame_buffer[:end + 2]
                beginning = self._video_frame_buffer.rfind(b'\xff\
                   xd8')
                if beginning == -1:
                    continue
```

```
                self.raw_video_frame = self._video_frame_buffer[
                    beginning:]
            except:
                if self.connected:
                    self.connected = False
                    if self.log_connection:
                        print('Camera DISCONNECTED')

    def get_frame(self):
        """
        Returns raw frame (bytes).
        :return: raw_frame or None
        """
        return self.raw_video_frame

    def get_cv_frame(self):
        """
        Returns decoded frame.
        :return: cv_frame or None
        """

        frame = self.get_frame()
        if frame is not None:
            frame = cv2.imdecode(np.frombuffer(frame, dtype=np.
                uint8), cv2.IMREAD_COLOR)
        return frame


class VideoStream:
    def __init__(self, logger=True):
        self.camera = Camera(log_connection=logger)
        self.logger = logger
        self._vidio_stream = None
        self._stop = threading.Event()
        self._stop.set()

    def start(self):
        if not self._stop.is_set():
            return
        self._stop.clear()
```

```
        self._vidio_stream = threading.Thread(target=self._stream,
            daemon=True)
        self._vidio_stream.start()

    def stop(self):
        self._stop.set()

    def _stream(self):
        """
        Continuously receives JPEG frames from ESP32, parses them,
            and renders it using standard OpenCV's image rendering
        facilities.
        """
        keymap = {"esc": 27}
        while True:
            key = cv2.waitKey(1)
            if key == keymap["esc"] or self._stop.is_set():
                cv2.destroyAllWindows()
                break

            # Try to receive a frame
            camera_frame = self.camera.get_frame()

            if camera_frame is None:
                if self.logger:
                    print("No frame")
                continue

            # Decode and render JPEG frame
            camera_frame = cv2.imdecode(np.frombuffer(camera_frame
                , dtype=np.uint8), cv2.IMREAD_COLOR)
            cv2.imshow('pioneer_camera_stream', camera_frame)
```

Listing 20: Полный сценарий отслеживания человека

```
import cv2
import mediapipe as mp
import numpy as np
from pioneer_sdk import Pioneer
from cam1 import Camera
from collections import namedtuple
import time
```

```
# создание объектов для работы нейросети:
# для рисования
mpDrawings = mp.solutions.drawing_utils
# предварительный конфигуратор детектора
skeletonDetectorConfigurator = mp.solutions.pose
# создание детектора с некоторыми настройками
skDetector = skeletonDetectorConfigurator.Pose(static_image_mode=
    False,
                                                    min_tracking_confidence
                                                        =0.5,
                                                    min_detection_confidence
                                                        =0.5,
                                                    model_complexity=1)

# использование встроенной камеры или камеры квадрокоптера
useIntegratedCam = False

# создание источников видео в зависимости от переменной
if not useIntegratedCam:
    pioneer = Pioneer(logger=True,log_connection=True)
    pioneer_cam = Camera()
else:
    cap = cv2.VideoCapture(0)

# объявление переменных, хранящих ширину и высоту изображения
IMGW, IMGH = None, None

# объявление переменной, хранящей значение нажатой кнопки на клави
    атуре
key = -1

# объявление переменной, хранящей время начала отсчета таймера для
     фотографирования
# то есть текущее время на момент прихода команды на создание фото
    графии
take_photo_time = -1

# объявление переменной для хранения времени последней отправки ко
    манды движения
```

```
# чтобы не спамить командами
async_waiter = -1

# объявление переменной, хранящей время начала отсчета таймера для
    следующего детектирования жестов
# то есть текущее время на момент детектирования жеста, для создан
   ия небольшой задержки до след. срабатывания
pose_detected = -1

# переменные, хранящие линейные скорости квадрокоптера
vx = .0
vy = .0
vz = .0
vr = np.radians(0)

# переменные для работы ПД регулятора при повороте
yaw_err = 0
yaw_errold = 0
yaw_kp = .04
yaw_kd = .02
yaw_k = 0.1

# переменные для работы ПД регулятора при движении вверх/вниз
z_err = 0
z_errold = 0
z_kp = 0.008
z_kd = 0.001

# переменные для работы ПД регулятора при движении вперед/назад
y_err = 0
y_errold = 0
y_kp = 10
y_kd = 2

# имена частей тела с индексами точек, образующих их
JOINTS_LIST = {"neck": [33, 0],
               "left_clavicle": [33, 12],
               "left_arm": [12, 14],
               "left_forearm": [14, 16],
               "right_clavicle": [33, 11],
               "right_arm": [11, 13],
```

```
                "right_forearm": [13, 15]}

# массив Точка имеет 4 именованных параметра, описывающих точку ск
    елета
Point = namedtuple('Point', 'x y z visibility')

# массив, содержащий сгенерированные части тела в виде векторов,
# в котором к элементам можно обратиться через точку
Parts = namedtuple("Parts", JOINTS_LIST.keys())

# массив, описывающий конкретную часть тела в виде вектора
Part = namedtuple("Part", 'x y angle')

# объявление функции remap
def remap(oldValue, oldMin, oldMax, newMin, newMax):
    """
    Функция для преобразования числовых значений из одного диапазо
        на в другой
    """
    oldRange = (oldMax - oldMin)
    if (oldRange == 0):
        newValue = newMin
    else:
        newRange = (newMax - newMin)
        newValue = (((oldValue - oldMin) * newRange) / oldRange) +
            newMin
    return newValue
# конец объявления

# объявление функции convert_points
def convert_points(points):
    """
    Функция для конвертации определенных нейросетью точек скелета
    из относительных координат (0-1.0) в абсолютные координаты
    """
    # массив, в котором будут храниться конвертированные точки
    converted_points = []

    # генерация базовой точки (между лопатками)
    # x - (x левой лопатки + x правой лопатки) / 2
    # y - (y левой лопатки + y правой лопатки) / 2
```

```
    # остальное по аналогии
    base_point = Point(x=round(IMGW * (points[12].x + points[11].x
        ) // 2),
                       y=round(IMGH * (points[12].y + points[11].y
                           ) // 2),
                       z=(points[12].z + points[11].z) / 2,
                       visibility=(points[12].visibility + points
                           [11].visibility) / 2)
    # непосредственная конвертация координат точек путем умножения
    # относительной координаты на ширину(высоту) изображения
    # (z и видимость остаются без изменений)
    for p in points:
        converted_points.append(Point(x=round(IMGW * p.x),
                                      y=round(IMGH * p.y),
                                      z=p.z,
                                      visibility=p.visibility))
    converted_points.append(base_point)
    return converted_points
# конец объявления


# объявление функции ang
def ang(v1):
    """
    Функция рассчитывает направление вектора на плоскости и возвра
        щает угол от 0 до 359
    """
    angle = round(np.degrees(np.arctan2(v1[1], -v1[0])))
    angle = remap(angle, -180, 179, 0, 359)
    return round(angle)
# конец объявления


# объявление функции generate_parts_vectors
def generate_parts_vectors(pts):
    """
    Функция для представления частей тела в виде векторов.
    Принимает набор точек, а возвращает вектора.
    """
    j = {}
    # проход по элементам словаря с именами и точками частей тела
```

```
    for joint in JOINTS_LIST.items():
        # 2 точки, образующие часть тела
        pos = joint[1]
        # из переданного набора найденных точек скелета выбираются
            эти 2 и считается вектор
        vec_x = pts[pos[1]].x - pts[pos[0]].x
        vec_y = pts[pos[1]].y - pts[pos[0]].y
        # сохранение вектора с именем части тела
        j.update({
            joint[0]: Part(vec_x, vec_y, ang([vec_x, vec_y]))
        })
    # конвертация в массив, к элементам которого можно обращаться
       через точку
    j = Parts(**j)
    return j
# конец объявления


# объявление функции eq
def eq(num1, num2, err=15):
    """
    функция для сравнивания двух чисел с погрешностью
    """
    if num1 < 0 or num2 < 0:
        return True
    if abs(num1-num2) <= err:
        return True
    elif (360-num1) <= err and abs(360 - num1 - num2) <= err:
        return True
    else:
        return False
# конец объявления


# объявление функции eq_all
def eq_all(lside=[], rside=[], neck=[]):
    """
    функция для быстрого сравнения всех прописанных векторов
    """
    ans = True
    if lside:
```

```
        # print('Left: ', parts.left_clavicle.angle, parts.
            left_arm.angle, parts.left_forearm.angle)
        ans = eq(parts.left_clavicle.angle, lside[0]) and \
              eq(parts.left_arm.angle, lside[1]) and \
              eq(parts.left_forearm.angle, lside[2]) and \
              ans
    if rside:
        # print("Right: ", parts.right_clavicle.angle, parts.
            right_arm.angle, parts.right_forearm.angle)
        ans = eq(parts.right_clavicle.angle, rside[0]) and \
              eq(parts.right_arm.angle, rside[1]) and \
              eq(parts.right_forearm.angle, rside[2]) and \
              ans
    if neck:
        ans = eq(parts.neck.angle) and ans
    # print('\n')
    return ans
# конец объявления


'''~~~
```

```
        # frame = cv2.resize(frame, (640, 480))

    # отзеркаливание изображения
    frame = cv2.flip(frame, 1)

    # получение размеров изображения
    IMGW = np.shape(frame)[1]
    IMGH = np.shape(frame)[0]

    # определение точек скелета
    detected_skeletons = skDetector.process(frame)
    # mpDrawings.draw_landmarks(frame, detected_skeletons.
        pose_landmarks, skeletonDetectorConfigurator.
        POSE_CONNECTIONS)
    # detected_skeletons = None

    # проверка, найдены ли точки
    if detected_skeletons and detected_skeletons.pose_landmarks is
        not None:
        # запись всех точек в переменную с более коротким именем
        points = detected_skeletons.pose_landmarks.landmark

        # конвертация координат точек из относительных в абсолютны
            е
        # (из диапазона от 0 до 1 в диапазон от 0 до ширины/высоты
            )
        converted_points = convert_points(points)
        # представление частей тела в виде векторов
        parts = generate_parts_vectors(converted_points)

        # отрисовка базовой точки (между лопатками)
        cv2.circle(frame, (converted_points[33].x,
            converted_points[33].y), 4, (255, 0, 0), 3)

        # проверка, был ли найден жест
        # необходима для создания задержки (промежутка) между дете
            ктированием
        if pose_detected == -1:
            # проверка направлений векторов и выявление поз
            # крещенные руки на груди
            if eq_all(lside=[180, 270, 45], rside=[0, 270, 135]):
```

```
            print("POSE 1")
            if take_photo_time == -1:
                print("Picture will be saved in 5 seconds")
                take_photo_time = time.time()
            pose_detected = time.time()

        # левая рука вбок
        elif eq_all(lside=[180, 180, 180]):
            print("POSE 2")
            vx = 0.7
            pose_detected = time.time()

        # правая рука вбок
        elif eq_all(rside=[0, 0, 0]):
            print("POSE 3")
            vx = -0.7
            pose_detected = time.time()

        # левая рука вбок и согнута в локте вверх
        elif eq_all(lside=[180, 180, 90]):
            print("POSE 4")
            vy = 1
            pose_detected = time.time()

        # правая рука вбок и согнута в локте вверх
        elif eq_all(rside=[0, 0, 90]):
            print("POSE 5")
            vy = -1
            pose_detected = time.time()

        if eq_all(lside=[180, 180, 270], rside=[0, 0, 270]):
            print("POSE 6")
            pose_detected = time.time()
            # если используется НЕ встроенная в компьютер каме
                ра
            if not useIntegratedCam:
                pioneer.land()

    # если с момента обнаружения жеста прошло 10мс - сбросить
        переменную, блокирующую детектирование
    if pose_detected != -1 and time.time() - pose_detected >
```

```
        0.1:
            pose_detected = -1

    # если время вызова таймера существует и уже прошло больше
        5 секунд, то сохранить фото
    if take_photo_time != -1 and time.time() - take_photo_time
        > 5:
            cv2.imwrite("image.png", frame)
            take_photo_time = -1
            pose_detected = -1

    # проверяем, что с последней отправки команды задачи скоро
        сти прошло больше 10мс
    if async_waiter != -1 and time.time() - async_waiter >
        0.1:
            async_waiter = -1

    # если используется НЕ встроенная в компьютер камера
    if not useIntegratedCam and async_waiter == -1:
        # если сконвертированные точки существуют, то работают
            регуляторы:
        if converted_points:
            # регулятор для удержания человека в центре изобра
                жения по рысканью (вращение вокруг своей оси)
            yaw_err = -(IMGW // 2 - converted_points[33].x) *
                yaw_k
            yaw_u = yaw_kp * yaw_err - yaw_kd * (yaw_err -
                yaw_errold)
            yaw_errold = yaw_err

            # регулятор определенного расстояния до человека п
                о оси Y (вперед/назад)
            y_err = -(-0.15 - converted_points[33].z)
            y_u = y_kp * y_err - y_kd * (y_err - y_errold)
            y_errold = y_err

            # регулятор для удержания человека в центре изобра
                жения по оси Z (вверх/вниз)
            z_err = ((IMGH * 1) // 2 - converted_points[33].y)
            z_u = z_kp * z_err - z_kd * (z_err - z_errold)
            z_errold = z_err
```

```
                # обновление переменных, содержащих значения (коор
                    динаты) для удержания коптером
                vr += -yaw_u
                vx += vx
                vy += 0
                vz += z_u
                pioneer.set_manual_speed_body_fixed(vx, vy, vz, vr
                    )

                async_waiter = time.time()

        # отрисовка всех точек и линий средствами используемой биб
            лиотеки
        mpDrawings.draw_landmarks(frame, detected_skeletons.
            pose_landmarks, skeletonDetectorConfigurator.
            POSE_CONNECTIONS)

    frame = cv2.resize(frame, (1920, 1080))

    # создание окна с изображением
    cv2.imshow("frame", frame)

    # считывание идентификатора нажатой кнопки с клавиатуры
    key = cv2.waitKey(1)

    # выход из программы при нажатии кнопки q
    if key == ord('q'):
        # если используется НЕ встроенная в компьютер камера
        if not useIntegratedCam:
            pioneer.disarm()
        break

    # посадка при нажатии кнопки l
    if key == ord('l'):
        # если используется НЕ встроенная в компьютер камера
        if not useIntegratedCam:
            pioneer.land()
            cordX, cordY = 0, 0
            cordZ = 1.5
```

```
    # взлет при нажатии кнопки j
    if key == ord('j'):
        # если используется НЕ встроенная в компьютер камера
        if not useIntegratedCam:
            pioneer.arm()
            pioneer.takeoff()
            pioneer.go_to_local_point(0, 0, 1.5, 0)

    if key == ord('u'):
        if not useIntegratedCam:
            print('Going up....')
            pioneer.go_to_local_point_body_fixed(x=0, y=0, z=0.5,
                yaw=0)

# завершение работы захвата изображений с камеры
if useIntegratedCam:
    cap.release()
# закрытие всех окон
cv2.destroyAllWindows()
```

9.3 Педагогический разбор

- Блок распознавания поз выделяет набор управляющих жестов.
- Блок регуляторов стабилизирует положение по yaw, z, y.
- Блок безопасности содержит явные клавиши j/l/q для взлёта, посадки и выхода.

Внимание

Этот сценарий относится к продвинутому уровню: перед запуском необходима проверка свободной зоны, стабильного освещения и готовности аварийного вмешательства оператора.

Задание 9. Минимальный набор упражнений

1. Отключите управление дроном и запустите только визуализацию скелета.
2. Измените коэффициенты yaw_kp, yaw_kd и оцените колебания.

3. Добавьте фильтрацию жестов по времени срабатывания.

9.4 Критерии успешности

1. Скелет детектируется в реальном времени без срывов отображения.
2. Минимум два жеста корректно распознаются и приводят к ожидаемой реакции.
3. Управляющий контур не вызывает неустойчивых рывков.

10 Уроки 13–14: многодронные сценарии и итоговый проект

10.1 Цели уроков

- Понять организацию роя через потоки и события.
- Научиться синхронизировать этапы взлёта и переходов по точкам.
- Спроектировать итоговую миссию с критериями приёмки.

10.2 Опорные листинги

Listing 21: Синхронный полёт двух дронов

```
__author__ = "Peter Bozhko"
__version__ = "1.0.0"
__maintainer__ = "Peter Bozhko"
__email__ = "p.bozhko@geoscan.ru"

import time
import threading
from pioneer_sdk import Pioneer

# IP-адреса дронов
mini_ip = ["192.168.137.62", "192.168.137.245"]

# Словари для хранения объектов
mini = {}  # объекты Pioneer
threads = {}  # объекты потоков для каждого дрона
point_reached = {}  # флаг долёта до точки от каждого дрона
tasks = {}  # задание (точка) для каждого дрона
event = threading.Event()  # Ивент для синхронизации полёта
event_finish = threading.Event()  # Ивент окончания задания

logger = False  # логгирования действий

# Координаты точек маршрута

coords = [[0, 1, 1],
          [0, 0, 1],
```

```
            [0, 1, 1]]


#  Функция для логгирования сообщений в консоль
def log(msg):
    if not logger:
        return
    print(f"Logger: {msg}")


# Функция ожидания соединения со всеми дронами
def wait_connection():
    while True:
        ret = True
        for ip in mini_ip:
            if not mini[ip].connected():
                ret = False
        if ret:
            return
        time.sleep(0.5)


#  Функция ожидания долёта всех коптеров до точки
def wait_task():
    global point_reached
    while True:
        count = 0
        for ip in mini_ip:
            if point_reached[ip]:
                count += 1
        log(f"CHECK POINT {count} FROM {len(point_reached)}")
        if count == len(point_reached):
            return
        time.sleep(0.3)


# Функция для перемещения дронов в начальную точку
def go_to_start_point(drone_ip):
    point_reached[drone_ip] = False  # Выставляем флаг долёта до т
        очки в False
    event.wait()  # Ожидаение команды на старт
```

```
    log(f"[{drone_ip}] ARM")
    mini[drone_ip].arm()
    log(f"[{drone_ip}] TAKE_OFF")
    mini[drone_ip].takeoff()
    log(f"[{drone_ip}] GO_TO_START_POINT")
    point = tasks[drone_ip]  # Берём точку из словаря tasks
    log(f"[{drone_ip}] GO_TO {point}")

    mini[drone_ip].go_to_local_point(x=point[0], y=point[1], z=
        point[2], yaw=0)
    while not mini[drone_ip].point_reached():
        pass
    point_reached[drone_ip] = True  # После прилёта в точку отмеча
        емся в словаре point_reached
    log(f"[{drone_ip}] POINT_REACHED ")
    run_mission_flight(drone_ip)  # Запуск основного задания


# Функция для выполнения полетного задания по заданным точкам
def run_mission_flight(drone_ip):
    while True:
        event.wait()  # Ожидания разрешения на полёт
        point_reached[drone_ip] = False  # Выставляем флаг долёта
            до точки в False
        if event_finish.is_set():  # Проверка флага конца миссии
            break
        point = tasks[drone_ip]  # Получаем следующую точку
        log(f"[{drone_ip}] GO_TO {point}")

        mini[drone_ip].go_to_local_point(x=point[0], y=point[1], z
            =point[2], yaw=0)
        while not mini[drone_ip].point_reached():
            pass
        point_reached[drone_ip] = True  # После прилёта в точку от
            мечаемся в словаре point_reached
    mini[drone_ip].land()  # После окончания миссии выполняется по
        садка


# Функция для экстренной посадки всех дронов
def all_land():
```

```
    for ip in mini_ip:
        mini[ip].land()
    return


#  Основная часть программы
def program():
    for ip in mini_ip:  # Создание отдельных потоков выполнения дл
       я каждого дрона
        print("Thread " + ip)
        threads[ip] = threading.Thread(target=go_to_start_point,
           args=[ip])
    for thread in threads.values():  # Запуск созданных потоков
        thread.start()
    input("ARM? ")  # Ожидание команды на взлёт от пользователя
    for ip in mini_ip:
        tasks[ip] = [0, 0, 1]  # Установка стартовой позиции для к
           аждого коптера
    event.set()
    event.clear()
    wait_task()
    input("START MISSION? ")  # Ожидание команды на старт миссии о
       т пользователя
    for coord in coords:
        for ip in mini_ip:  # Установка следующей точки для каждог
           о коптера
            tasks[ip] = coord
        event.set()  # Флаг разрешения на полёт (синхронизация)
        event.clear()
        time.sleep(0.3)
        wait_task()  # Ожидание прилёта всех дронов
    event_finish.set()  # Флаг окончания полёта
    event.set()


if __name__ == '__main__':
    # # Инициализация объектов Pioneer для каждого дрона
    for ip in mini_ip:
        mini[ip] = Pioneer(ip=ip, name=ip, logger=False)
    wait_connection()  # Ожидание соединения со всеми дронами
    try:
```

```
        program()  # Запуск программы
    except BaseException:  # В случае ошибки или остановки пользов
        ателем посадка дронов
        all_land()
```

Listing 22: Синхронный полёт с PiZero и MAVLink-портом

```
__author__ = "Peter Bozhko"
__version__ = "1.0.0"
__maintainer__ = "Peter Bozhko"
__email__ = "p.bozhko@geoscan.ru"

import time
import threading
from pioneer_sdk import Pioneer

# IP-адреса дронов
pioneer_ip = ["192.168.137.62", "192.168.137.245"]

# Словари для хранения объектов
pioneer = {}          # объекты Pioneer
threads = {}          # объекты потоков для каждого дрона
point_reached = {}    # флаг долёта до точки от каждого дрона
tasks = {}            # задание (точка) для каждого дрона

event = threading.Event()          # Ивент для синхронизации полёта
event_finish = threading.Event()   # Ивент окончания задания
logger = False                     # логгирования действий

# Координаты точек маршрута
coords = [[0, 1, 1],
          [0, 0, 1],
          [0, 1, 1]]

# Функция для логгирования сообщений в консоль
def log(msg):
    if not logger:
        return
    print(f"Logger: {msg}")

# Функция ожидания соединения со всеми дронами
def wait_connection():
```

```
    while True:
        ret = True
        for ip in pioneer_ip:
            if not pioneer[ip].connected():
                ret = False
        if ret:
            return
        time.sleep(0.5)

# Функция ожидания долёта всех коптеров до точки
def wait_task():
    global point_reached
    while True:
        count = 0
        for ip in pioneer_ip:
            if point_reached[ip]:
                count += 1
        log(f"CHECK POINT {count} FROM {len(point_reached)}")
        if count == len(point_reached):
            return
        time.sleep(0.3)

# Функция для перемещения дронов в начальную точку
def go_to_start_point(drone_ip):
    point_reached[drone_ip] = False  # Выставляем флаг долёта до т
        очки в False
    event.wait()  # Ожидание команды на старт
    log(f"[{drone_ip}] ARM")
    pioneer[drone_ip].arm()
    log(f"[{drone_ip}] TAKE_OFF")
    pioneer[drone_ip].takeoff()
    log(f"[{drone_ip}] GO_TO_START_POINT")
    point = tasks[drone_ip]  # Берём точку из словаря tasks
    log(f"[{drone_ip}] GO_TO {point}")
    pioneer[drone_ip].go_to_local_point(x=point[0], y=point[1], z=
        point[2], yaw=0)
    while not pioneer[drone_ip].point_reached():
        pass
    point_reached[drone_ip] = True  # После прилёта в точку отмеча
        емся в словаре point_reached
    log(f"[{drone_ip}] POINT_REACHED ")
```

```
    run_mission_flight(drone_ip)  # Запуск основного задания

# Функция для выполнения полетного задания по заданным точкам
def run_mission_flight(drone_ip):
    while True:
        event.wait()  # Ожидания разрешения на полёт
        point_reached[drone_ip] = False  # Выставляем флаг долёта
           до точки в False
        if event_finish.is_set():  # Проверка флага конца миссии
            break
        point = tasks[drone_ip]  # Получаем следующую точку
        log(f"[{drone_ip}] GO_TO {point}")
        pioneer[drone_ip].go_to_local_point(x=point[0], y=point
           [1], z=point[2], yaw=0)
        while not pioneer[drone_ip].point_reached():
            pass
        point_reached[drone_ip] = True  # После прилёта в точку от
           мечаемся
    pioneer[drone_ip].land()  # После окончания миссии выполняется
        посадка

# Функция для экстренной посадки всех дронов
def all_land():
    for ip in pioneer_ip:
        pioneer[ip].land()
    return

# Основная часть программы
def program():
    for ip in pioneer_ip:  # Создание отдельных потоков выполнения
        для каждого дрона
        print("Thread " + ip)
        threads[ip] = threading.Thread(target=go_to_start_point,
           args=[ip])
    for thread in threads.values():  # Запуск созданных потоков
        thread.start()

    input("ARM? ")  # Ожидание команды на взлёт от пользователя
    for ip in pioneer_ip:
        tasks[ip] = [0, 0, 1]  # Установка стартовой позиции для к
           аждого коптера
```

```
    event.set()
    event.clear()
    wait_task()

    input("START MISSION? ")  # Ожидание команды на старт миссии о
       т пользователя
    for coord in coords:
        for ip in pioneer_ip:  # Установка следующей точки для каж
           дого коптера
            tasks[ip] = coord
        event.set()  # Флаг разрешения на полёт (синхронизация)
        event.clear()
        time.sleep(0.3)
        wait_task()  # Ожидание прилёта всех дронов

    event_finish.set()  # Флаг окончания полёта
    event.set()

if __name__ == '__main__':
    # Инициализация объектов Pioneer для каждого дрона
    for ip in pioneer_ip:
        pioneer[ip] = Pioneer(ip=ip, mavlink_port=5656, name=ip,
           logger=False)
    wait_connection()  # Ожидание соединения со всеми дронами
    try:
        program()  # Запуск программы
    except BaseException:  # В случае ошибки или остановки пользов
       ателем посадка дронов
        all_land()
```

Listing 23: Круговой роевой полёт с фазовым сдвигом

```
__author__ = "Peter Bozhko"
__version__ = "1.0.0"
__maintainer__ = "Peter Bozhko"
__email__ = "p.bozhko@geoscan.ru"

import math
import time
import threading
import random
from pioneer_sdk import Pioneer
```

```
# IP-адреса дронов
mini_ip = ["192.168.137.144", "192.168.137.228", "192.168.137.203"
    ]
# mini_ip = ["192.168.137.199", "192.168.137.4"]

# Словари для хранения объектов
mini = {}  # объекты Pioneer
threads = {}  # объекты потоков для каждого дрона
offsets = {}  # смещение коптеров на окружности относительно перво
    го
point_reached = {}  # флаг долёта до точки от каждого дрона
tasks = {}  # задание (точка) для каждого дрона
event = threading.Event()  # Ивент для синхронизации полёта
event_finish = threading.Event()  # Ивент окончания задания

logger = True  # логгирования действий

# Параметры для полёта по окружности
r = 0.75  # радиус окружности
x_c = 0  # x координата центра окружности
y_c = 1.5  # y координата центра окружности
z_c = -0.8  # z координата центра окружности
circle_points = 12  # количество точек на окружности
step = 2 * math.pi / circle_points  # шаг в радианах между точками
max_points = 13  # количество точек в задании


# Функция расчёта координаты каждого коптера
def circle_coord(index, counter):
    x = x_c + r * math.cos(step * counter + offsets[index])
    y = y_c + r * math.sin(step * counter + offsets[index])
    z = z_c - (0.2 * (random.randint(0, 2)))  # высота задаётся ра
        ндомно для каждой точки
    return [x, y, z]


#  Функция для логгирования сообщений в консоль
def log(msg):
    if not logger:
        return
```

```
    print(f"Logger: {msg}")


# Функция ожидания соединения со всеми дронами
def wait_connection():
    while True:
        ret = True
        for ip in mini_ip:
            if not mini[ip].connected():
                ret = False
        if ret:
            return
        time.sleep(0.5)


#  Функция ожидания долёта всех коптеров до точки
def wait_task():
    global point_reached
    while True:
        count = 0
        for ip in mini_ip:
            if point_reached[ip]:
                count += 1
        log(f"CHECK POINT {count} FROM {len(point_reached)}")
        if count == len(point_reached):
            return
        time.sleep(0.3)


# Функция для перемещения дронов в начальную точку
def go_to_start_point(drone_ip):
    point_reached[drone_ip] = False  # Выставляем флаг долёта до т
       очки в False
    event.wait()  # Ожидаение команды на старт
    log(f"[{drone_ip}] ARM")
    mini[drone_ip].arm()
    log(f"[{drone_ip}] TAKE_OFF")
    mini[drone_ip].takeoff()
    log(f"[{drone_ip}] GO_TO_START_POINT")
    point = tasks[drone_ip]  # Берём точку из словаря tasks
    log(f"[{drone_ip}] GO_TO {point}")
```

```
    mini[drone_ip].go_to_local_point(x=point[0], y=point[1], z=
        point[2], yaw=0)
    while not mini[drone_ip].point_reached():
        pass
    point_reached[drone_ip] = True  # После прилёта в точку отмеча
        емся в словаре point_reached
    log(f"[{drone_ip}] POINT_REACHED ")
    run_mission_flight(drone_ip)  # Запуск основного задания


# Функция для выполнения полетного задания по заданным точкам
def run_mission_flight(drone_ip):
    while True:
        event.wait()  # Ожидания разрешения на полёт
        point_reached[drone_ip] = False  # Выставляем флаг долёта
            до точки в False
        if event_finish.is_set():  # Проверка флага конца миссии
            break
        point = tasks[drone_ip]  # Получаем следующую точку
        log(f"[{drone_ip}] GO_TO {point}")

        mini[drone_ip].go_to_local_point(x=point[0], y=point[1], z
            =point[2], yaw=0)
        while not mini[drone_ip].point_reached():
            pass
        point_reached[drone_ip] = True  # После прилёта в точку от
            мечаемся в словаре point_reached
    mini[drone_ip].land()  # После окончания миссии выполняется по
        садка


# Функция для экстренной посадки всех дронов
def all_land():
    for ip in mini_ip:
        mini[ip].land()
    return


#  Основная часть программы
def program():
```

```
    for ip in mini_ip:  # Создание отдельных потоков выполнения дл
        я каждого дрона
        print("Thread " + ip)
        threads[ip] = threading.Thread(target=go_to_start_point,
            args=[ip])
        offsets[mini_ip.index(ip)] = mini_ip.index(ip) * (2 * math
            .pi) / len(mini_ip)
    for thread in threads.values():  # Запуск созданных потоков
        thread.start()
    input("ARM? ")  # Ожидание команды на взлёт от пользователя
    for i in range(0, len(mini_ip)):
        tasks[mini_ip[i]] = circle_coord(i, 0)  # Установка старто
            вой позиции для каждого коптера
    event.set()
    event.clear()
    wait_task()
    input("START MISSION? ")  # Ожидание команды на старт миссии о
        т пользователя
    for k in range(1, max_points):  # количество точек
        for i in range(0, len(mini_ip)):  # Установка следующей то
            чки для каждого коптера
                tasks[mini_ip[i]] = circle_coord(i, k)
        event.set()  # Флаг разрешения на полёт (синхронизация)
        event.clear()
        time.sleep(0.3)
        wait_task()  # Ожидание прилёта всех дронов
    event_finish.set()  # Флаг окончания полёта
    event.set()


if __name__ == '__main__':
    # # Инициализация объектов Pioneer для каждого дрона
    for ip in mini_ip:
        mini[ip] = Pioneer(ip=ip, name=ip, logger=False)
    wait_connection()  # Ожидание соединения со всеми дронами
    try:
        program()  # Запуск программы
    except BaseException:  # В случае ошибки или остановки пользов
        ателем посадка дронов
        all_land()
```

10.3 Итоговый проект

ТЗ проекта

Команда разрабатывает миссию из трёх этапов: взлёт, фигура по точкам, посадка. На каждом этапе фиксируются телеметрия и один критерий безопасности.

Этап	Проверяемый результат	Критерий зачёта
Старт миссии	Все аппараты перешли в состояние полёта	Нет рассинхронизации старта более 2 с
Маршрут	Прохождение всех точек	Не менее 90% точек достигнуто
Завершение	Штатная посадка	Нет аварийных выключений

Table 3: Рубрикатор итоговой практики

10.4 Контрольные вопросы

1. Для чего в роевых скриптах используется threading.Event?
2. Почему в учебном коде нежелателен while not point_reached(): pass без паузы?
3. Какие данные нужно логировать для разбора инцидентов?

11 План-проспект главы по программированию на Python

Логика построения

План построен по траектории: базовые операции полёта → автономные фигуры → телеметрия и камера → компьютерное зрение → интеграционный проект.

Поурочный конспект

№	Цель урока	Необходимые знания	Ключевые команды	Минимальные упражнения	Критерии успеха
1	Подключение и базовый цикл	Запуск Python-скрипта	Pioneer(), close_connection()	Запуск test.py	Есть стабильно соединени
2	Арминг, взлёт, посадка	try/finally	arm(), takeoff(), land()	Полный цикл взлёт-посадка	Посадка б аварии
3	Диагностика батареи и дальномера	while, sleep	get_battery_status(), get_dist_sensor_data()	Цикл с выводом показания заряда	Индикаци совпадает порогами
4	Полёт по скорости	Время и интервалы	set_manual_speed()	Прямой участок 1–2 с	Устойчиво движение
5	Полёт по локальным точкам	СК x,y,z	go_to_local_point(), point_reached()	Маршрут из 3 точек	Все точки достигнут
6	Базовые фигуры	Тригонометрия окружности	math.sin, math.cos	Квадрат и окружность	Фигура замкнута
7	Интерактивный режим	Клавиатурные события	send_rc_channels()	Настройка WASD-контроля	Команды соответств клавишам

8	Работа с видеопотоком	OpenCV-окно	Camera(), get_cv_frame()	Вывод кадров и выход по ESC	Поток без зависаний
9	Калибровка камеры	Шахматная доска	calibrateCamera, FileStorage	Получение data.yml	Файл коэффици создан
10	Детекция ArUco	Базовый CV-пайплайн	ArucoDetector	Детекция меток в кадре	Метка стабильно распознаё
11	Координаты ArUco и управление	Геометрия PnP	solvePnP(), set_manual_speed_body_fixed()	Удержание дистанции до маркера	Нет рывко и потерь устойчиво
12	Трекинг человека	Скелетная модель	MediaPipe Pose, ПД-регулятор	Распознавание 2 жестов	Корректн реакция н жест
13	Роевой полёт	Потоки и события	threading.Event	Синхронный старт 2 дронов	Рассинхро менее 2 с
14	Итоговый проект	Проектирование миссии	Комбинация изученного API	Полная миссия с отчётом	Пройдены критерии рубрики

Форма отчётности

- Логи запуска и завершения каждого урока.
- Скриншоты ключевых моментов: маршрут, детекция, итоговая посадка.
- Самооценка по критериям урока: зачёт/незачёт с коротким пояснением.

12 Приложение

12.1 Алфавитный индекс ключевых команд

Команда	Назначение
arm()	Включение моторов перед стартом
disarm()	Выключение моторов после посадки или в экстренной ситуации
get_battery_status()	Получение напряжения батареи
get_cv_frame()	Получение кадра камеры в формате OpenCV
get_dist_sensor_data()	Чтение дальномера
go_to_local_point()	Переход к точке в локальной системе координат
land()	Штатная посадка
point_reached()	Проверка достижения целевой точки
send_rc_channels()	Отправка значений RC-каналов для ручного управления
set_manual_speed()	Управление линейными скоростями и рысканьем
set_manual_speed_body_fixed()	Управление скоростями в связанной системе координат
takeoff()	Автоматический взлёт

12.2 Карта скриптов для преподавателя

Скрипт	Где использовать в курсе
mission_examples/PiZero_test.py	Урок 2: первый взлёт и посадка
mission_examples/set_manual_speed.py	Урок 4: прямолинейный полёт
mission_examples/go_to_local_point.py	Урок 5: полёт по точкам
mission_examples/circle_flight.py	Урок 6: базовые фигуры
camera_examples/camera_calibration.py	Урок 9: калибровка
aruco_examples/aruco_flight.py	Урок 11: визуальная обратная связь
PioneerHumanTracking/human-tracking.py	Урок 12: продвинутый CV
group_flight_examples/swarm_OPT.py	Урок 13: роевые алгоритмы

Table 6: Быстрый выбор скрипта под учебную цель

References

1. Camera calibration with OpenCV. — 2026. — URL: https://docs.opencv.org/4.x/dc/dbb/tutorial_py_calibration.html (visited on 03/04/2026).
2. MediaPipe Pose. — 2026. — URL: https://developers.google.com/mediapipe/solutions/vision/pose_landmarker (visited on 03/04/2026).
3. Slatkin B. Effective Python : 90 Specific Ways to Write Better Python. — 2nd ed. — Addison-Wesley Professional, 2019.